COMP10020 · INTERNET TECHNOLOGIES · GROUP B

Supabase

PostgreSQL and backend services for web applications



Vladyslav Papin
[Banner ID]

[Name 2]
[Banner ID]

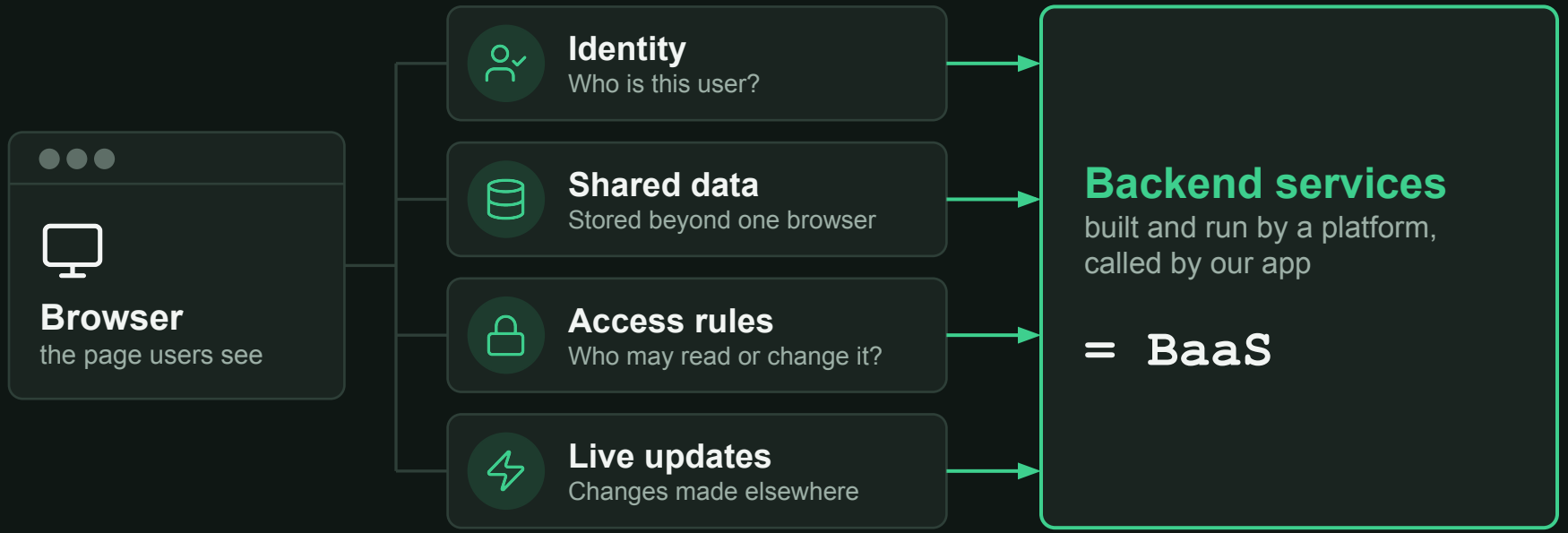
[Name 3]
[Banner ID]

Stanley Khumalo
[Banner ID]

[Name 5]
[Banner ID]

Why a frontend needs a backend

BaaS provides reusable backend services



Supabase starts with PostgreSQL

Open-source services around one database · hosted or self-hosted

Auth
sign-in · sessions

Data API
PostgREST · HTTP

Realtime
changes via WebSocket

PostgreSQL
the core: tables, data and row-level security

Storage
not used in this example

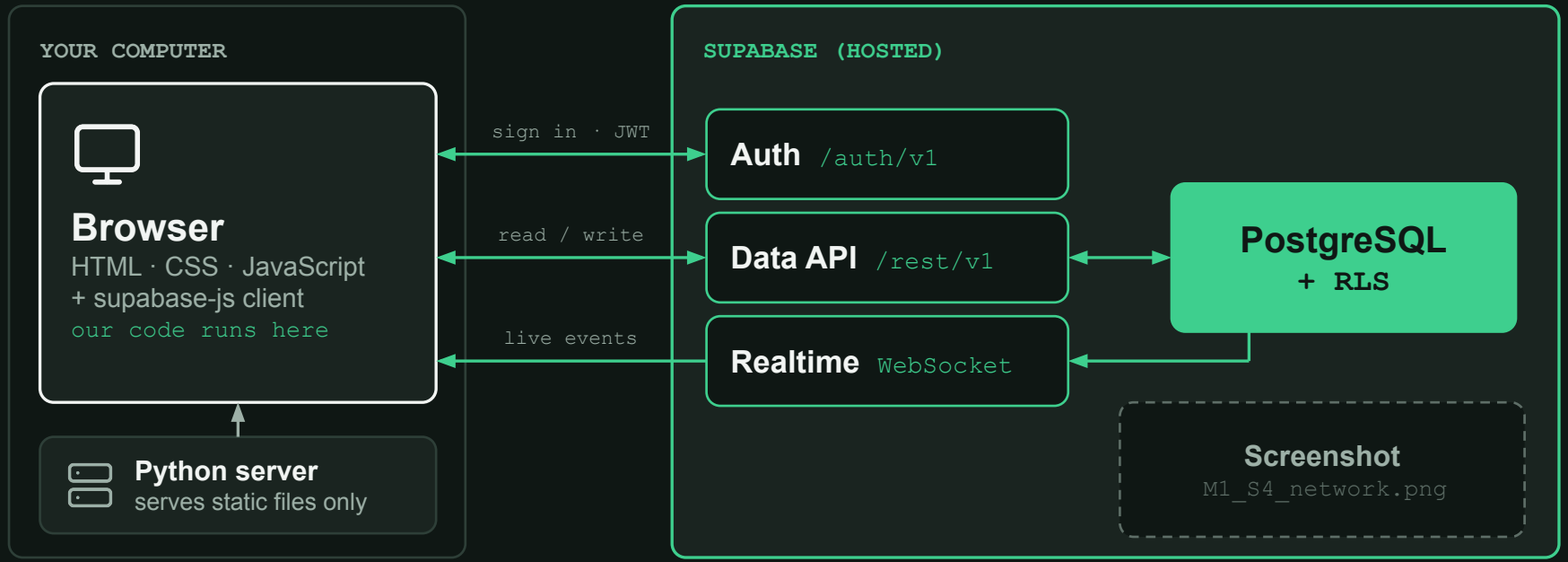
Edge Functions
not used in this example

Studio
dashboard · not in the request flow

Screenshot
`M1_S3_services.png`
`dashboard sidebar`

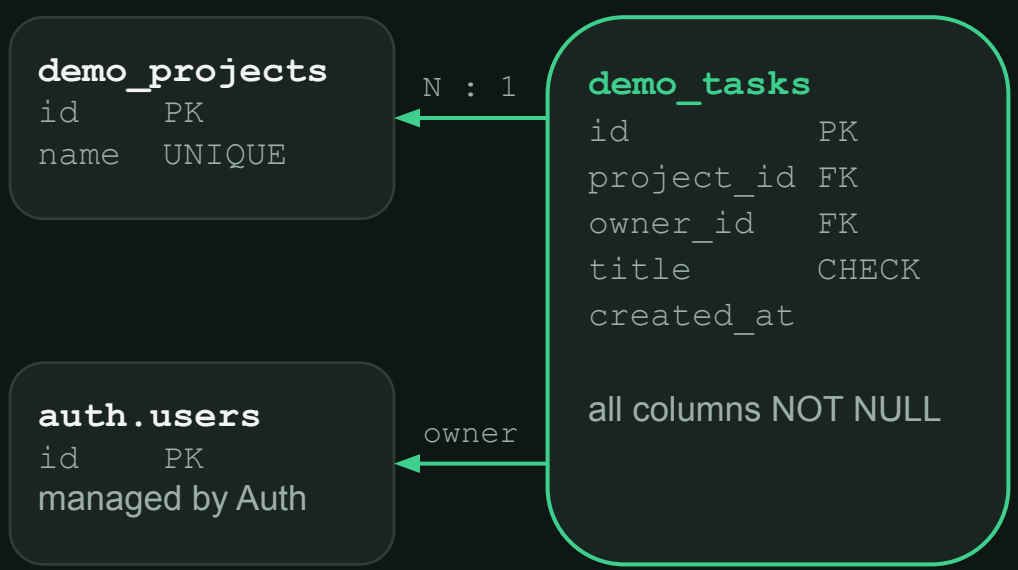
Where our code runs

Python serves files. Supabase provides the backend.



Relational data

Tables, keys and constraints describe the data. PostgreSQL enforces them.



M2_S5_schema.png
Schema Visualizer

Data API

PostgREST turns tables into a REST API; supabase-js builds the request.

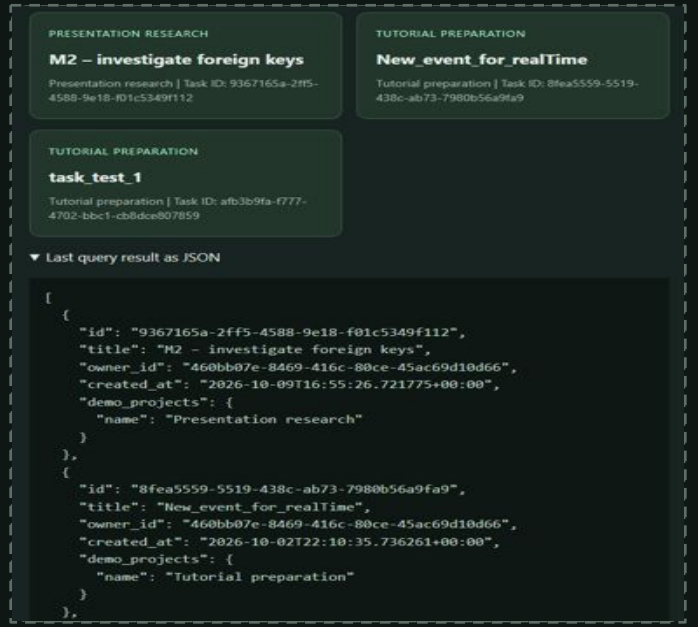
app.js · loadTasks() (simplified)

```
db.from('demo_tasks')
  .select('id, title, demo_projects(name)')
  .order('created_at', { ascending: false })
```

GET /rest/v1/demo_tasks

```
[{ "title": "M2 – investigate foreign keys",
   "demo_projects": {
     "name": "Presentation research" } }]
```

The foreign key lets the API embed the project name, so one request returns both tables.



Data constraints

PostgreSQL itself rejects a bad reference: integrity, not access control.

Insert task
`project 0000…`

→

PostgreSQL
foreign key check

→

✕ 23503
no row saved

23503 · integrity

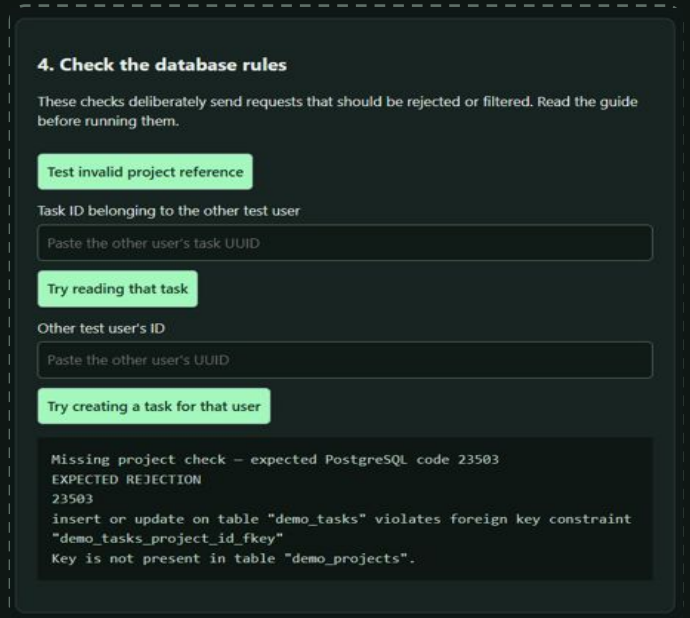
Does the referenced row exist?

Same answer for every user.

42501 · access (RLS)

Is this user allowed to do it?

Depends on who is signed in.



Identity and permission

Authentication says who you are; authorisation decides what you may do.

AUTHENTICATION · Supabase Auth

Sign in
email + password

→

Supabase Auth
/auth/v1/token

↓

JWT (access token)
sub = user id · role = authenticated

↓

Authorisation inside PostgreSQL: every API request carries the JWT; the database reads it as auth.uid() and applies grants and RLS.

M3_S8_users.png
Authentication → Users

M3_S8_identity.png
Current identity

Database access

Grants open the table; RLS policies choose the rows.

```
-- 1. Grant: which operations
grant select, insert on public.demo_tasks
  to authenticated;
-- 2. RLS: which rows
create policy
  "Users read their own demo tasks"
  on public.demo_tasks for select
  to authenticated
  using ((select auth.uid()) = owner_id);
create policy
  "Users insert their own demo tasks"
  on public.demo_tasks for insert
  to authenticated
  with check ((select auth.uid()) = owner_id);
```

Key points

- No UPDATE or DELETE grant for the page
- Signed-out users get no table access
- Publishable key ≠ admin key
- Hiding a button is not access control

Access proof

User B cannot read or forge A’s tasks. The database refuses, not the page.

FOUR CHECKS

Check	Result	Proves
Own task insert	✓ saved	allowed
Bad project ID	23503	integrity
Read A’s task ID	[] no rows	RLS (read)
Forge A’s owner	42501	RLS (write)

An empty result alone proves nothing: the task ID must exist and be visible to A.

M3_S10_read.png
No rows returned

M3_S10_write.png
EXPECTED REJECTION · 42501

Push and polling

Realtime pushes changes over one WebSocket instead of asking repeatedly.

POLLING



“Anything new?” every few seconds, mostly empty

PUSH · SUPABASE REALTIME



One open WebSocket, an event only on INSERT

```
alter publication supabase_realtime
  add table public.demo_tasks;
```

M4_S11_publication.png

supabase_realtime publication

Event flow

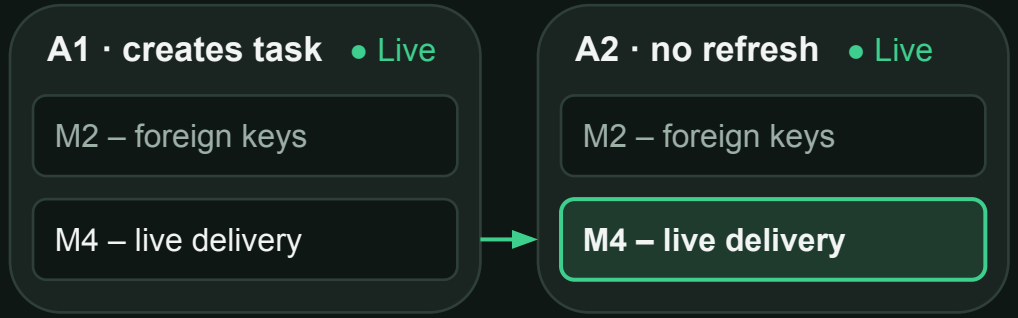
Each event passes the same RLS rules, then the page reads fresh data.

1. **INSERT** window A1 saves a task
2. **PostgreSQL** the row is stored
3. **Realtime** captures the change
4. **RLS check** may A2 see this row?
5. **Callback** event count +1
6. **Fresh SELECT** tasks with the project name
7. **Render** card + highlight (local CSS)



Two windows

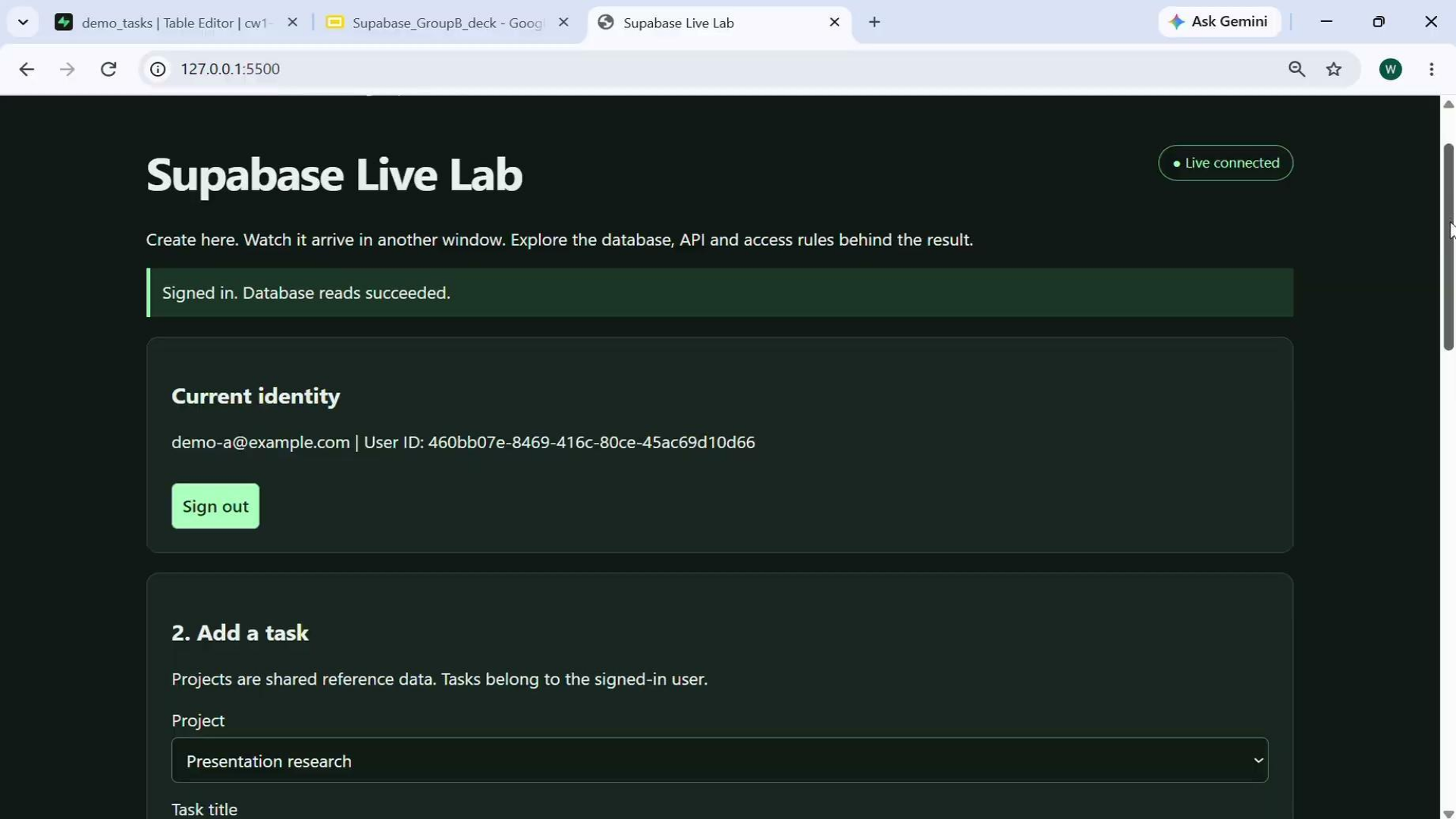
The second window updates without Refresh. Our example handles INSERT only.



M4_S13_live.gif
second window updates

- ✓ The card appears in A2; live event count +1
- ✓ User B gets no event for A's task

Note: edits and deletes are not subscribed in v2.1


demo_tasks | Table Editor | cw1-
Supabase_GroupB_deck - Goog
Supabase Live Lab
Ask Gemini
127.0.0.1:5500
Supabase Live Lab
• Live connected
Create here. Watch it arrive in another window. Explore the database, API and access rules behind the result.
Signed in. Database reads succeeded.
Current identity
demo-a@example.com | User ID: 460bb07e-8469-416c-80ce-45ac69d10d66
Sign out
2. Add a task
Projects are shared reference data. Tasks belong to the signed-in user.
Project
Presentation research
Task title

Work and cost

Supabase removes server work, not design work. Cost depends on usage.

Supabase provides

- Hosted PostgreSQL
- Auth and JWT sessions
- REST Data API
- Realtime server

We still build

- Schema and constraints
- Grants and RLS policies
- Frontend (HTML/JS)
- Access-rule tests

Free $0 · Pro from $25 a month

supabase.com/pricing, checked 10 Oct 2026. Free: 500 MB database, 50,000 MAU; projects pause after 1 week idle.

M5_S15_pricing.png
pricing, with date

M5_S15_usage.png
project Usage

Verdict

A strong fit for data-centred web apps, if the team owns SQL and RLS.

	Supabase (hosted)	Firebase	Custom backend + PostgreSQL	Self-hosted Supabase
Database	PostgreSQL	Firestore (NoSQL)	PostgreSQL	PostgreSQL
Access rules	RLS in database	Security Rules	Our API code	RLS in database
Setup	Fast: managed	Fast: managed	Slow: build API	Medium: run stack
Lock-in	Low–medium	High: Google only	Lowest	Low

Good fit: auth, CRUD and live updates; prototypes and small teams

Weaker fit: heavy server logic, strict control, a team new to SQL

SOURCES

References

Official documentation, accessed October 2026. Links are on our website.

Supabase documentation

- Architecture
- API keys
- Auth
- Row Level Security
- Data REST API
- Realtime: Postgres Changes
- Edge Functions
- Self-hosting with Docker
- Pricing (checked 10 October 2026)

PostgreSQL, PostgREST and Firebase

- PostgreSQL: Constraints
- PostgreSQL: Row Security Policies
- PostgreSQL: Error Codes
- PostgREST documentation
- Firebase: SQL Connect
- Firebase Blog: Data Connect GA (April 2025)
- Firebase: Cloud Firestore
- Firebase: Security Rules

GROUP B · SUPABASE

Thank you

Questions?

Website & tutorial: **papinvlad.github.io/supabase-site**
Repository: **github.com/PapinVlad/supabase-site**



Slides, tutorial and references

Supabase or Firebase?

Both are managed backends. They differ in data model, access rules and hosting.

Supabase

- PostgreSQL: tables, joins and SQL
- Access rules inside the database (RLS)
- Open source; can be self-hosted
- Data exports with pg_dump

Firebase

- Firestore: NoSQL documents and collections
- Access rules in Firebase Security Rules
- Google-hosted; emulators are for testing
- SQL Connect adds PostgreSQL via GraphQL

Why we chose Supabase: our data is relational, and the access rules sit next to it as SQL.

Is the key in our page a leak?

No. The publishable key identifies the project; RLS decides what each user can do.

Key	Postgres role	RLS	Where it belongs
Publishable sb_publishable_…	anon or authenticated	**Applies**	Browser, mobile app, source code
Secret sb_secret_…	service_role	**Bypassed**	Servers only; browsers get 401

Legacy equivalents are the anon and service_role JWT keys, which Supabase is deprecating by the end of 2026.

The risk is a missing policy, not a visible key.

What about edits and deletes?

Our example handles INSERT only. UPDATE and DELETE need three changes.

1 Grant

```
grant update, delete on public.demo_tasks
  to authenticated;
```

2 Policies

```
create policy … for update
  using ((select auth.uid()) = owner_id)
  with check ((select auth.uid()) = owner_id);
```

3 Subscribe

```
.on('postgres_changes',
  { event: '*', schema: 'public', table: 'demo_tasks' }, …)
```

A delete policy works the same way, with using (…) only.

Realtime caveat: Supabase does not apply RLS to DELETE events. Our fresh SELECT after each event still returns only permitted rows.

Can we leave Supabase?

Yes: the data is PostgreSQL, but self-hosting moves operations work to us.

Take the data

- supabase db dump or pg_dump
- Restore into another PostgreSQL

Run it ourselves

- Docker Compose setup
- Same open-source Auth, PostgREST, Realtime, Storage and Studio

What we then own

- Security and secrets
- HTTPS and email (SMTP)
- Updates and backups

Supabase-specific: auth.users, auth.uid() in policies and supabase-js calls need rework elsewhere.

Server logic and growth

Edge Functions run server-side code; Broadcast handles heavy live traffic.

Edge Functions

- Server-side TypeScript on Deno
- Run on edge nodes close to the user
- Hold secrets, such as payment keys
- Short tasks; long jobs belong in workers

Growing traffic

- Realtime checks each event for every subscriber
- Over ~3,000 subscribers to the same changes: use Broadcast
- Free: 500 MB, 50,000 MAU, pauses after a week idle
- Pro from $25 a month, plus usage

Our example needs neither, but both are available without leaving Supabase.